

Firestore 스키마 설계

레거시 19개 표 **240개 컬럼 전부**가 어디로 갔는지 한 줄씩 적는다. 무엇을 **지우고 · 바꾸고 · 새로 넣었는지**가 이 문서의 전부다.

갱신 2026-09-06 원본 Aimbroad_database_schema-240812 대상 app/types/schema.ts

00 목표 — 스키마는 목적이 아니라 수단이다

지금 이 흐름에서 가운데 사람 손을 얹어는 것이 전부다.

현재

DID 입력 → MySQL → 사람이 추출·엑셀 가공 → CSV → JaionX admin 업로드

목표

DID 입력 → Firestore → 파이썬 자동 변환 → JaionX 갱신

→ JaionX 데이터가 지금과 동일해지는 순간, 구 DID 폐기

규칙 0 — 다른 모든 원칙보다 위에 있다

JaionX가 받는 모든 컬럼은 Firestore 데이터만으로 복원 가능해야 한다. 복원 불가능한 값이 하나라도 있으면 그 값은 반드시 저장한다.

"계산으로 대체한다"는 결정은 복원 가능성이 증명될 때만 유효하다.

상태 표기 — 이 문서 전체에서 쓰는 6가지

표기	뜻
유지	이름 그대로 옮김
이름변경	값은 같고 이름만 바뀜 (접두사 제거 / 신용어)
삭제 (계산)	저장 안 함. 필요할 때 records 에서 계산 — 1,200건에 0.16ms
삭제 (경로)	Firestore 경로가 그 역할을 대신함
삭제 (불필요)	새 구조에서 존재 이유가 없어짐
신규	레거시에 없던 것

01 두 갈래 — 입력 데이터 / 저장 관리 데이터

둘 다 Firestore에 영구 저장된다. 구분 기준은 저장 여부가 아니라 누가 만들고 얼마나 자주 바뀌는가다.

입력 데이터 총이 곧 저장소다

“입력 데이터”는 임시 데이터가 아니라 **입력을 통해 만들어지는 데이터**다. 경기에서 입력한 사실은 그 문서 자체로 영구 저장되며, 별도 저장 관리 층으로 옮겨 담지 않는다. 기존 명칭은 유지한다.

예를 들어 아래 문서는 **사카가 이 경기 전반 32분에 옐로카드를 받았다는 사실**이다.

```
matches/{gm_id}/recordings/H/cards/abc123
{ playerId: '1218', half: 'H1', halfSeconds: 1920, card: 'Y' }
```

경기별 카드 기록은 저장한다. “이번 시즌 누적 3장” 같은 누적값은 해당 시즌의 카드 기록을 조회·집계해 계산한다. 경기가 추가되거나 원본이 정정될 때 누적값을 따로 맞추는 구조를 만들지 않는다(원칙 3). 확정 시점의 스냅샷은 기존 원칙대로 별도 보존한다.

타입

경로

① 입력 데이터 스키마 — 기록자가 실시간으로 만든다 —————

MatchDoc	matches/{gm_id}	
RecordingDoc	matches/{gm_id}/recordings/{H A}	★
RecordDoc	matches/{gm_id}/recordings/{H A}/records/{id}	
PathSnapshotDoc	.../paths/{pathId}	[확정 시점에만]
PlayerStatsDoc	.../playerStats/{playerId}	[확정 시점에만]

② 저장 관리 데이터 스키마 — 관리자가 화면에서 입력한다 ————

LeagueDoc	leagues/{id}	
SeasonDoc	seasons/{id}	
StadiumDoc	stadiums/{id}	
TeamDoc	teams/{teamId}	
TeamSeasonEntry	teams/{teamId}/seasons/{seasonId}	승강제 대응
SquadEntry	teams/{teamId}/squad/{playerId}	[파생 뷰]
PlayerDoc	players/{playerId}	
ContractDoc	players/{playerId}/contracts/{id}	★ 이적시장 원장
RecorderProfileDoc	recorders/{uid}	

① 입력 데이터

② 저장 관리 데이터

저장 여부

Firestore 영구 저장

Firestore 영구 저장

누가 만드나

기록자 (앱)

관리자 (관리 화면)

얼마나 늘어나나

경기당 **400~1,200건**씩 계속

거의 안 바뀜

핵심 문제

유실 방지 · 정합성

이력 관리 · 감사

읽기 방식

실시간 구독

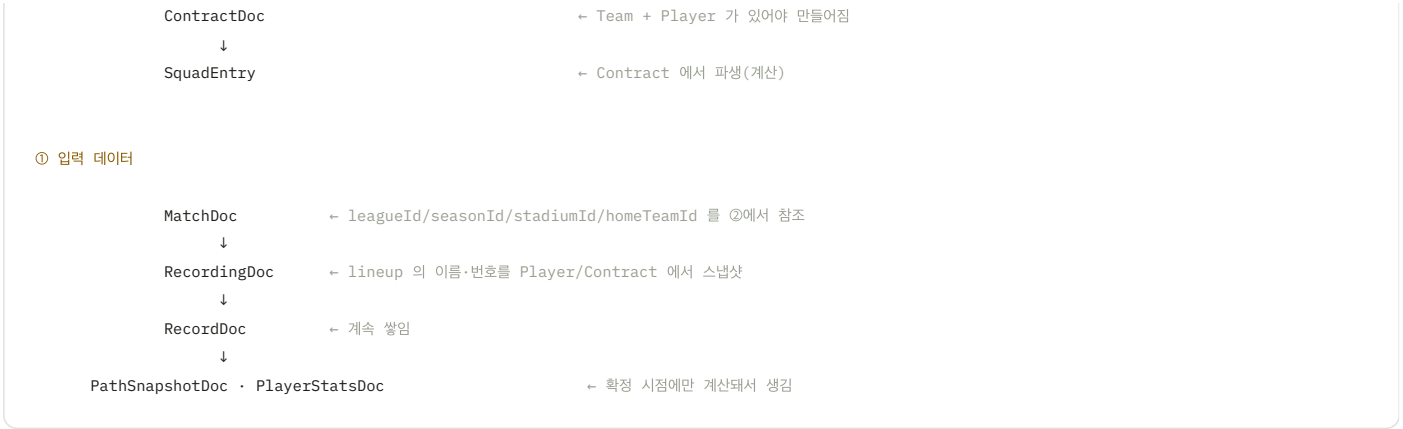
파이썬 API + 스냅샷 (§12)

02 무엇부터 생산되는가

문서형이라도 참조 관계는 그대로 있다. ②가 없으면 ①이 채울 값이 없다.

② 저장 관리 데이터 — 뿌리부터

LeagueDoc · SeasonDoc · StadiumDoc · TeamDoc · PlayerDoc ← 뿌리, 서로 독립
↓



참조 · 스냅샷 · 자체생성 — 세 가지가 섞여 있다

구분	예	원본이 바뀌면
참조	MatchDoc.homeTeamId = 'ARSX'	같이 바뀜 (ID만 들고 있음)
스냅샷	lineup['1218'].name = 'B. Saka'	안 바뀜 — 그 시점 사실로 박제 (원칙 7)
자체생성	RecordDoc.area / act / res	원본이 없음 — 사람이 그 자리에서 만들

선수가 이적하거나 등번호를 바뀌도 **작년 경기 기록의 이름·번호는 그대로여야 한다**. 그래서 라인업만은 참조가 아니라 스냅샷이다.

03 MatchDoc — matches/{gm_id}

ff_game (16개 컬럼) → 일정. 두 기록 세션이 공유하는 사실만 담는다.

문서 ID는 gm_id 를 그대로 쓴다

matches/20262027ep1j2rgn13na28rd2b8q0001

20262027 ep1 j2rgn13na28rd2b8q 0001

시즌

리그

리그 고정 상수

일련번호

gm_id 는 JaionX의 모든 테이블을 잇는 조인 키다. 예시의 가운데 상수 구간은 2024-25-2025-26-2026-27 세 시즌 780건 전부 동일 — 시즌별 난수가 아니다.

GM_ID 는 우리가 만들지 않는다 — 사람이 입력한다

경기 일정 관리 화면(/manage/schedules)에서 담당자가 일정을 등록할 때 gm_id 를 직접 입력한다. 입력받은 문자열이 그대로 문서 ID가 된다.

따라서 우리는 채번 규칙을 알 필요도, 일련번호를 발급할 필요도 없다. 우리가 임의로 발급하면 같은 경기에 다른 번호가 붙어 JaionX 와 엉영 맞지 않는다.

담당자 입력 20262027ep1j2rgn13na28rd2b8q0001

↓

matches/20262027ep1j2rgn13na28rd2b8q0001

그래서 입력 화면에 반드시 필요한 것

오타 하나에 JaionX 매칭이 통째로 깨진다. 담당자가 직접 입력하는 값이므로 형식 검증 (해당 리그의 실제 ID 규격 · 시즌 · 일련번호)과 경기 정보 일치 확인이 필요하다. 예시 ID는 30자이므로 일괄 32자 검증을 적용하지 않는다.

그리고 Firestore 는 같은 문서 ID 로 쓰면 **말없이 덮어쓴다**. 실수로 같은 `gm_id` 를 두 번 등록하면 앞 경기가 사라진다 — **중복 확인** 이 반드시 있어야 한다.

레거시	새 필드	상태
gm_id	(문서 ID)	이름변경
gm_date	date	이름변경
gm_time	kickoffTime	이름변경
gm_h_t_code / gm_a_t_code	homeTeamId / awayTeamId	이름변경
gm_league	leagueId	이름변경
gm_round	round	이름변경
gm_s_code	stadiumId	이름변경
gi_goal_home / gi_goal_away	score.home / score.away	이름변경 — 각 recording 이 자기 쪽만 merge
gm_state	—	삭제(불필요) — recordings.status 가 대신함
is_realtime	recordings.inputMode	이동 — 경기 속성이 아니라 기록자별 설정
is_old / is_view / is_onair	—	삭제(불필요)
gm_sub_league	—	⚠ 미결정 (§14)
—	seasonId	NEW — 다만 gm_id 앞 8자리와 중복 이다. 질의 편의용
—	createdAt / updatedAt	NEW — 레거시 ff_game 에는 없다. 관례로 붙인 것

SROUND 는 DID 저장 스키마에서 제외한다

matches 와 recordings 모두에 저장하지 않는다. 경기 식별은 gm_id , 홈/원정 구분은 recordings/{H|A} 가 맡는다. JaionX 출력의 S-Round 처리는 §04에 정리했다.

04 RecordingDoc — recordings/{H|A} ★

ff_game_info (49개 컬럼) → 팀별 기록 세션. 이 구조는 레거시가 이미 하고 있던 것을 보존한 것이다.

```
// lib/extra.lib.php:3799 - 경기 하나에 gi 행이 정확히 2개
$gi[$row['gi_write_code']] = $row;
if (count($gi) < 2) return FALSE; // "양팀 입력 모두 종료되지 않은 경기는 진행 불가"
```

KPI 집계가 전부 ff_game_info 에 있는 것도 그래서다 — KPI는 경기 단위가 아니라 팀-기록자 단위다. 바뀐 것은 표현 방법뿐이다: JOIN → 경로, gi_write_code 컬럼 → 문서 ID.

식별 · 상태

레거시	새 필드	상태
gi_id	legacyGiId	삭제(경로) + 동기화 매핑용으로만 보존
gm_id	—	삭제(경로) — 부모가 이미 가리킴
gi_write_code	(문서 ID) + side	이름변경 — 중복 세션이 구조적으로 불가능해짐
gi_user_id	recorders / recorderIds	단수 → 복수 (2인 기록)
gi_h_t_code / gi_a_t_code	teamId / opponentTeamId	이름변경
gi_state	status	이름변경 — 000/H1B/H1E → ready/H1/H1_done
gi_part	fieldSide	이름변경
gi_formation	formationKey	이름변경 — 좌표는 상수, 문자열만 저장
gi_regdt / gi_moddt	createdAt / updatedAt	이름변경
gi_part_ex	—	⚠️ 미결정 — 연장전 진영 (§14)
is_old	—	삭제(불필요)
—	maxSeq	NEW 레코드 정렬키 발급
—	kpiComputedAt / syncedAt	NEW

S-Round — 표시·정렬용 출력값, DID 저장 필드에서는 제외

앞선 JaionX 확인 결과, admin/dMST.vue · admin/pMST.vue 에 표시 컬럼이 있고, composables/pMST.ts 는 S-Round로 정렬한다. build_MatchSchedule.py 는 dMST의 값을 읽어 일정 출력에 넣는다. 따라서 JaionX에서 전혀 쓰지 않는 값이라는 뜻은 아니다.

결정 — RECORDINGS.SROUND 를 제외하고 DID 채번 로직을 만들지 않는다

파이프라인의 경기 식별·홈·원정 매칭 기준은 gm_id + team_type 이다. 기존 S-Round 홀짝 매칭은 사용하지 않는다. DID에서는 문서 ID의 H/A를 출력 시 team_type으로 전달한다.

담당자는 gm_id 를 입력하고, DID는 그 값을 경기 문서 ID로 보존한다. sRound 추가 입력이나 클라이언트 채번은 도입하지 않는다.

후속 구현: JaionX에 필요한 S-Round는 파이썬의 dMST 출력 단계에서 처리한다. 앞서 3시즌 780경기로 검증한 규칙을 기반으로 gm_id-round-H/A에서 기존 값과 동일하게 복원되는지 출력 대조 검증을 연결한다. 일정 빌더가 dMST에서 값을 읽는 것과, 그 값을 생성하는 단계는 구분한다. 이 문서 수정으로 파이프라인 구현까지 완료된 것은 아니다.

시간 — 4개 컬럼이 맵 하나로

```
halves: {
  H1: { startedAt, seconds: 2712 },
  H2: { startedAt, seconds: 2840 },
}
```

레거시	새 필드	상태
gi_h1_begin ~ gi_h4_begin	halves.H1~H4.startedAt	이름변경 + 구조화
gi_h1_seconds ~ gi_h4_seconds	halves.H1~H4.seconds	이름변경 + 구조화
gi_seconds	—	삭제(계산) — halves 합계

45분을 넘겨도 HALF 는 안 바뀐다

전반 48:30 은 **H1** 이 계속 흐르는 것이지 H3 가 아니다. half 는 그대로 'H1' 이고 halfSeconds 만 2910 이 된다. H3/H4 는 진짜 연장전 에만 쓴다.

JaionX 5분 구간 컬럼도 JMX-H1-45-Plus 처럼 **45분 이후를 통 하나로** 묶는다 — 몇 분을 넘겼든 그 버킷이다.

KPI — 확정 시점에만 kpi 맵으로

레거시	새 필드	상태
gi_tmp	kpi.TAP	이름변경(신용어)
gi_tap	kpi.DAP	이름변경(신용어)
gi_ctp	kpi.DTP	이름변경(신용어)
gi_ttp / gi_bap / gi_asr / gi_ssr	kpi.TTP / BAP / ASR / SSR	유지 — 단 SSR 산식은 (GOAL-OG) / SHOT
gi_sht / gi_gol	kpi.SHOT / kpi.GOAL	이름변경
gi_ctb / ctm / cta / cts	kpi.B / M / A / S	이름변경(신용어 DTB/DTM/DTA/DTS)
—	kpi.OG	NEW 자책골 수 (D-MST 요구)
gi_utp / gi_stp / gi_csp	—	삭제(계산) — paths 에서 셈
gi_gtb / gi_gtm / gi_gsr	—	삭제(계산)
gi*_sc / _sr / _pr (9개)	—	삭제(계산) — 성공수·성공률은 원자값 나눗셈 한 번
gi_score	—	삭제(이동) — 팀평점은 jpd-rating(대외비)

lineup — ff_game_player 의 출전 정보가 여기로

18명을 항상 통째로 읽고 쓴다. 서버컬렉션이면 로드마다 18 read, map 이면 1 read다.

레거시	새 필드	상태
gp_type	type ('START'/'BENCH')	이름변경
gp_order	order	이름변경
gp_in_half / gp_in_half_seconds	inHalf / inSeconds	이름변경
gp_out_half / gp_out_half_seconds	outHalf / outSeconds	이름변경

레거시	새 필드	상태
gp_in_seconds / gp_out_seconds / gp_seconds	—	삭제(계산) — 경기 기준 초·출전시간
gp_point_plus / gp_point_minus	—	⚠ 미결정 — 상단 -3/-1/+1/+3 버튼 (\$14)
—	slot	NEW 포메이션 슬롯 ID
—	no / name / pos	SNAPSHOT 이적해도 과거 기록 불변

ff_game_player_log — 별도 컬렉션을 만들지 않는다

레거시	새 필드	상태
pl_out_p_id / pl_in_p_id	lineup[선수].outHalf / inHalf	병합 — 맵 키가 곧 선수
pl_in_half / pl_in_seconds	inHalf / inSeconds	병합
pl_in_position	pos	병합
pl_in_order	—	⚠ 미결정 — 교체 순번 (\$14)
gi_id / t_code / pl_regdt / pl_moddt	—	삭제(경로) / 삭제(불필요)

교체는 기록된다 — 저장 형태만 다르다

교체 정보는 lineup 맵 안에서 완결되고, 어차피 recordings 문서를 통째로 읽으므로 추가 읽기가 0이다. 별도 subs 컬렉션을 두면 오히려 질의가 한 번 더 불는다.

교체 목록 화면은 lineup 에서 in/out 이 채워진 항목만 골라 시간순 정렬하면 된다 — 저장은 한 벌, 화면은 필요한 만큼.

05 RecordDoc — records/{id}

ff_game_record (37개 컬럼) → 플레이 1건. 37개 중 17개가 사라진다 — 대부분 파생 플레그다.

레거시	새 필드	상태
gr_id	(문서 ID)	이름변경 — Firestore 자동 ID
gi_id	—	삭제(경로) — 1,200회 중복이 사라짐
t_code	—	삭제(경로) — recordings.teamId 에 있음
p_id	playerId	이름변경 — 'OWN' = 자책골
gr_half / gr_half_seconds	half / halfSeconds	이름변경 — 반드시 저장
gr_seconds	—	삭제(계산) — halves + halfSeconds
gr_area_code	area	이름변경
gr_act_code / gr_res_code	act / res	이름변경 — GB/GX/GOAL 신 코드

레거시	새 필드	상태
gr_pos_x / gr_pos_y	posX / posY	이름변경 + 단위 변환 — 레거시는 971×634 픽셀, 우리는 0~100% (아래 참고)
gr_shoot_pos_x/y	shootPosX / shootPosY	이름변경
gr_part_pos_x/y	—	삭제(불필요) — posX/posY 로 통합
gt_id	—	삭제(계산) — export 시 computeAttackPaths() 가 채움
gr_is_* (14개)	—	삭제(계산) — flags 로 항상 파생
gr_area_code_org	—	미결정 (§14)
gr_shoot_rate_x/y	—	미결정 (§14)
gr_regdt	createdAt	이름변경
gr_moddt / is_old	—	삭제(불필요)
—	seq	NEW 같은 초 안의 순서
—	isShot	NEW write-back 후에도 쏘이었음 보존
—	shootDspRange	NEW 골포스트 1m 판정 캐시
—	bapReason	NEW ff_game_bap 병합
—	createdBy / playerIdBy	NEW 2인 기록 시 누가 넣었나
—	source / playerIdSource	NEW 사람 / 비전 구분

좌표 — 픽셀에서 백분율로

```
// 03.areacode.sql 첫 줄
Ground = 971 * 634 // 경기장 전체 크기

posX = gr_pos_x / 971 * 100
posY = gr_pos_y / 634 * 100
```

실데이터로 검증됐다. TACTIC.xlsx 의 어느 레코드가 gr_pos_x=327, gr_pos_y=598, area=7 인데, 03.areacode.sql 의 R07 범위가 X=319~484 / Y=452~634 로 정확히 그 안에 들어간다. 화면 해상도와 무관하게 이 971×634 가 좌표 원점 이다.

왜 gr_is_* 14개를 지우나

레거시가 저장한 이유	지금은?
계산 로직이 서버(PHP)에만 있었다	사라짐 — didLogic.ts 를 모든 화면이 공유
SQL 집계가 필요했다 (SUM(gr_is_tap))	사라짐 — 확정 스냅샷을 따로 저장
2013년 단말이 약했다	사라짐 — 실측 0.16ms

그리고 결정적으로, 레코드 하나를 고치면 앞뒤 루트의 플래그가 연쇄로 바뀐다. 시간 1건을 고치면 4건 × 14개 = 56개 값이 바뀐다. 수정 기능이 이미 있으므로 저장하면 반드시 어긋난다.

ff_game_bap — 표 자체를 없앤다

레거시	새 필드	상태
bap_id	—	삭제(불필요) — 레코드와 1:1이라 별도 ID가 필요 없다
gi_id / gr_id	—	삭제(경로) — 레코드 자신이 곧 그것
(BAP 여부·사유)	RecordDoc.bapReason	병합
gr_half / gr_half_seconds / gr_seconds	—	삭제(중복) — 레코드에 이미 있음

`computeBap()` 은 **레코드 1개당 이벤트 0~1개**를 산출한다. `gt_id` (여러 레코드를 하나로 묶음)와 달리 그룹핑이 아니므로, 별도 문서 대신 레코드 자신에게 표시하면 된다.

06 확정 스냅샷 — paths · playerStats

파생 스냅샷은 기록 중에는 저장하지 않는다. 입력 원본은 계속 저장한다. status:'final' 이 될 때 한 번에 배치로 쓴다.

시점	동작
기록 중	파생 스냅샷은 저장하지 않는다. 원본 records 는 저장하며 화면 지표는 즉석 계산
확정	paths / playerStats / kpi 를 한 번에 배치 write
확정 후 수정	스냅샷 무효화 → 재계산 → 덮어쓰기

변동기와 확정기의 요구가 반대이기 때문이다. 기록 중에는 매 탭마다 값이 바뀌므로 저장이 곧 부채고, 확정 후에는 리포트·동기화가 "그때 값"을 안정적으로 요구한다.

ff_game_path (11) → PathSnapshotDoc

레거시	새 필드	상태
gt_id	gtId	이름변경
gt_upp / gt_utp / gt_ctp / gt_stp	ptype	구조 변경 — 불린 4개 → 단일 열거값
gt_csp	dsp	이름변경(신용어)
gt_ttp	ttp	유지
gt_res_code	resCode	이름변경
gi_id / gt_regdt / gt_moddt	—	삭제(경로) / 삭제(불필요)
—	recordIds	NEW 이 루트에 속한 레코드

ff_game_player 개인 KPI → PlayerStatsDoc

46개 컬럼 중 **16개만 저장**한다. 저장하는 것도 전부 신용어로 바꾼다 — JaionX playerMST 는 아직 `gp_ctb` 같은 옛 이름을 쓰지만 **그건 내보낼 때의 이름이지 우리 저장소의 이름이 아니다.**

레거시	새 필드	상태
gp_tmp / gp_tap / gp_ctp	TAP / DAP / DTP	이름변경(신용어)
gp_ctb / ctm / cta / cts	DTB / DTM / DTA / DTS	이름변경(신용어)
gp_utp / ttp / gtb / gtm / asr / ssr	UTP / TTP / GTB / GTM / ASR / SSR	유지
gp_sht / gp_ast / gp_gol	SHOT / AST / GOAL	이름변경
gp*_sc / _sr / _tr / _pr (11개)	—	삭제(계산) — 성공수·성공률·시도율
gp_score_rel / abs / gp_score	—	삭제(이동) — 평점은 jpd-rating(대외비)만 채움

07 ② 팀 · 선수 · 계약

경기 기록이 아니라 관리 화면이 쓰는 참조 데이터. 입력·수정 대상이므로 이력과 감사 필드가 필요하다.

ff_team (14) → TeamDoc

레거시	새 필드	상태
t_code	(문서 ID)	이름변경
t_name / t_name_kr / t_name_full / t_name_short	name / nameKr / nameFull / nameShort	이름변경
u_text_color_code	textColor	이름변경
s_code	stadiumId	이름변경
t_coach / t_coach_kr	coach / coachKr	이름변경
t_begin / t_end	foundedAt / dissolvedAt	이름변경
l_code	currentLeagueId + teams/{id}/seasons/{id}	구조 변경 — 단일값이면 승강제를 표현 못 한다
t_name_ae / t_name_cn	—	 미결정 (§14)
—	crestUrl	NEW
—	createdAt/By · updatedAt/By	NEW 감사 필드

ff_player (9) → PlayerDoc — 정체성만

레거시	새 필드	상태
p_id	(문서 ID)	이름변경
p_name / p_name_en / p_name_full	name / nameEn / nameFull	이름변경
p_foot / p_birth / p_height / p_nation	foot / birth / height / nation	이름변경

레거시	새 필드	상태
p_id_old	legacyPlayerId	이름변경
—	active	NEW 은퇴/삭제 대신 비활성 — 과거 기록을 고아로 만들지 않는다
—	currentTeamId / currentNo / currentPos	파생 캐시 계약에서 갱신

어느 팀 소속인지는 시간에 따라 변하므로 여기 두지 않는다. 그건 계약 원장의 일이다.

ff_player_team (8) → ContractDoc — 이적시장 원장 ★

레거시	새 필드	상태
p_id	—	삭제(경로)
t_code	teamId	이름변경
pt_begin / pt_end	from / to	이름변경 — <code>to = null</code> 이면 현재 소속
pt_num / pt_pos	no / pos	이름변경 — 계약에 종속(팀 옮기면 바뀐다)
l_code	leagueId	이름변경
p_id_old	—	삭제(불필요) — PlayerDoc 에 있음
—	fromTeamId	NEW 직전 소속팀 — 한 문서로 "A→B" 완성
—	transferType	NEW TRANSFER / LOAN / LOAN_RETURN / FREE / YOUTH / RETIRE
—	fee / currency	NEW

이적 한 건 = 트랜잭션 하나 — ① 이전 계약 `to` 닫기 ② 새 계약 생성 ③ `players.current*` 갱신 ④ 양 팀 `squad` 갱신. **계약 문서가 곧 이적 이벤트다** — 별도 transfers 컬렉션을 두지 않는 이유다.

SquadEntry — 현재 스쿼드 **파생 뷰**

원장만 있으면 계약 30건 + 선수 30명 = **61 read + 조인**이 필요하다. 이 뷰가 있으면 **1 query**로 끝난다. 갱신은 이적 시에만(연 수십 회)이라 어긋날 위험이 낮다 — 원칙 3의 정당한 예외.

08 ② 리그 · 시즌 · 경기장 · 기록자

레거시 표	새 타입	주요 변경
ff_league (3)	LeagueDoc	league_code→(문서 ID), league_name→name, league_name_en→nameEn. NEW country
ff_season (7)	SeasonDoc	l_code→leagueId, s_fr_date/s_to_date→from/to. s_fr_dt/s_to_dt 삭제(중복) — 날짜/일시 이중 보관
ff_stadium (8)	StadiumDoc	s_hometeam→homeTeamId, s_ground→surface, 나머지 접두사 제거
ff_user (5)	RecorderProfileDoc	u_id→(Auth UID), u_hand→handedness. u_pw 삭제. NEW role, level

FF_USER 는 통째로 사라지지 않는다

로그인(`u_id` / `u_pw`) 기능만 Auth 로 넘어가고, 이름·소속팀·역할 같은 프로필 정보는 그대로 남는다. 형태만 `recorders/{uid}` 로 바뀐 것이다.

```
recorders/{uid}: {
  name, teamId, handedness,
  role: 'recorder' | 'admin' // 데이터 관리 권한
  level: 'basic' | 'advanced' // 갱신 버튼 구성 (§11)
}
```

09 표 자체를 없앤 것

19개 표 중 5개는 새 구조에서 존재 이유가 사라졌다.

레거시 표	왜 없었나
ff_game_bap	레코드와 1:1 이라 별도 ID·문서가 필요 없다 → <code>RecordDoc.bapReason</code> 으로 병합
ff_game_player_log	교체 정보가 <code>lineup</code> 맵 안에서 완결된다 → 추가 읽기 0
ff_game_state	관리자가 <code>recordings.status</code> 를 직접 고치는 것으로 대체
ff_session ff_member_session	세션ID·기기ID·GCM등록ID — Firestore Auth 토큰이 전부 대체 . 남길 값이 하나도 없다
ff_formation	포메이션은 팀 소유물이 아니라 공용 도형 이다. 4-4-2 슬롯 좌표는 어느 팀이 쓰든 같다 → 코드 상수로 두고 <code>formationKey</code> 문자열만 저장

아직 타입이 없는 것

ff_game_card (7개 컬럼) — 트리에는 `cards/{cardId}` 로 적어놨으나 `schema.ts` 에 타입이 없다. 경기별 사실은 `recordings/{H/A}/cards/{cardId}` 에 영구 저장한다.
CardDoc 타입과 저장 연동은 후속 구현 대상이다 (§14).

10 역할 분담 — 무엇을 어디서 계산하나

판정 로직의 권위는 파이썬에 있다. 클라이언트 TS 는 화면 보조일 뿐이다.

어디	프로젝트	하는 일	접근
Nuxt 클라이언트 <code>didLogic.ts</code>	jpd-did	루트가 끊겼는지 + 공격영역 진입 여부 → 선수 입력 버튼 띄우기 전용	누구나
KPI 서비스 <code>python</code>	jpd-did	DAP/DTP/STP/UTP 분류, B·M·A·S 역할, BAP, KPI 집계	개발자 포함
평점 서비스 <code>python</code>	jpd-rating 별도 프로젝트	선수평점·팀지수·등급, 기준값, 채번 규칙, dMST/playerMST 조립 및 JaionX 쓰기	1~2명

클라이언트에 남는 판정은 하나뿐이다 — "어느 레코드에 선수를 물어볼까". DTP/DAP라는 이름을 붙이는 일은 전부 **파이썬** 이 한다. 클라는 ① 루트가 끊겼나 ② 공격영역에 들어갔나(`area < 7`) 두 가지만 본다.

방식	경기당 선수 입력 요청
DAP 판정 사용 (현재)	457회
엑트마다 전부 물어봄	861회 - 거의 2배

DAP 판정이 곧 "선수를 물어볼 최소 집합"이다. 향후 비전이 등번호를 채우면 이 버튼 자체가 사라지고, 클라이언트 판정도 제거해 파이썬 단일 구조가 된다.

KPI 가 쌓이는 곳과 평점이 쌓이는 곳을 분리한다

평점 서비스만 KPI 를 읽는다. 반대 방향은 없다. KPI 프로젝트를 전부 뒤져도 DAP 65, BAP 7, SHOT 3 같은 숫자만 있고 점수는 없다. 기준값(ratingBaseline)도 평점 저장소에만 둔다.

다만 JaionX 가 입력과 출력을 같은 표에 보여주므로 **작성하고 회귀분석을 하면 공식은 복원된다**. 그러므로 API 호출 제한 같은 역산 방지 장치는 만들지 않는다 - 실익 없이 복잡도만 는다. 분리로 얻는 것은 **복사 경로 차단 · 예외 처리의 정확도 · 영업비밀 지위**이며, 코드를 어디 두느냐의 문제라 비용이 0이므로 그대로 한다.

11 갱신 트리거 - 계산은 오직 여기서만

원칙

레코드가 쌓이는 것과 계산이 도는 것은 별개다. 레코드는 계속 서버에 쌓이지만, 판정·KPI·평점은 갱신 트리거 없이는 절대 돌지 않는다.

버튼	KPI 서비스	평점 서비스	JAIONX
전반 데이터 갱신	판정 + KPI + 5분 구간 1~10	팀 JMX 구간 1~10	✗
후반 데이터 갱신	판정 + KPI + 5분 구간 1~20	팀 JMX 구간 1~20	✗
최종 데이터 갱신 & 경기 종료	전체 확정 집계	팀 지수 + 선수 평점 + 등급	✔ 전승

계정 등급 뱃지

	BASIC	ADVANCED
전반 · 후반 갱신	있음	없음
최종 갱신 & 경기 종료	있음	있음

basic 이 갱신을 누르면 그 half 의 수정도 함께 잠긴다. 해제는 기록자 화면이 아니라 /manage/locks (관리자 전용)에서만 한다 - 기록자 화면에 두면 본인이 스스로 풀 수 있어 잠금이 무의미해진다.

팀 JMX 는 5분 구간 시계열에서 나온다

```
for i in 1..20:
    # H1 0-5 ... H2 45+
    norm_k(i) = 누적_k(i) / (5분기준_k × i) × 100
```

```
avg(i) = mean(DAP, DTP, SHOT, SSR, GOAL)
JMX(i) = avg(i) × 0.6 + 40          # 구간별 JMX 시계열
```

최종 JMX 는 20번째 구간값일 뿐이다. 그래서 `half / halfSeconds` 가 필수다 - 반 구분과 반 기준 경과차가 없으면 이 시트를 만들 수 없다. 선수 평점의 5분 단위는 아직 없다(추후).

오프라인은 정상시 동작 방식이 아니다

기록은 보통 인터넷이 되는 곳에서 한다. 오프라인 지속성은 끊겼을 때 입력이 멈추지 않게 하는 안전장치이며 품질을 위한 것이다. 따라서 네트워크를 일부러 끊어두지 않는다 — 그래야 2인 기록이 두 모드 모두에서 성립한다.

오프라인은 정상시 동작 방식이 아니다

기록은 보통 인터넷이 되는 곳에서 한다. 오프라인 지속성은 끊겼을 때 입력이 멈추지 않게 하는 안전장치이며 품질을 위한 것이다. 따라서 네트워크를 일부러 끊어두지 않는다 — 그래야 2인 기록이 두 모드 모두에서 성립한다.

12 조회 층 – 화면은 Firestore 를 직접 나열하지 않는다

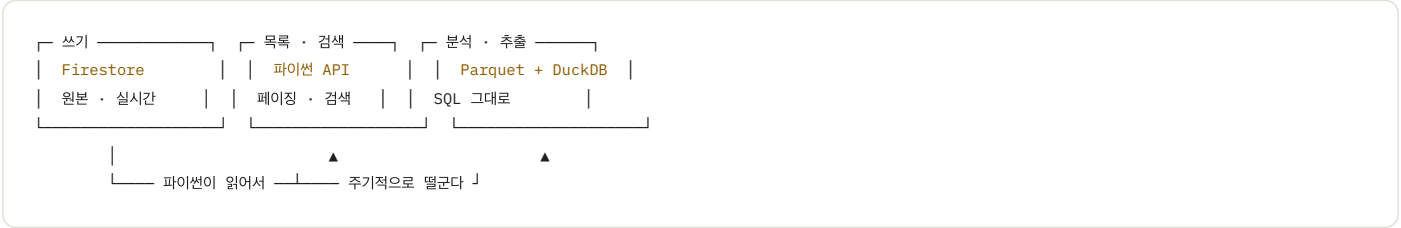
규칙

목록·검색·집계를 `getDocs` 로 컬렉션을 **출력**해서 **해결**하지 않는다. 이 규칙이 없으면 화면을 만들 때마다 전량 로드로 되돌아간다.

규칙

목록·검색·집계를 `getDocs` 로 컬렉션을 **출력**해서 **해결**하지 않는다. 이 규칙이 없으면 화면을 만들 때마다 전량 로드로 되돌아간다.

지금 footballX 가 겪는 문제가 그것이다 — dMST 760 read, playerMST **11,500 read**.
 Firestore 는 필드를 골라 받을 수 없고(웹 SDK 에 SELECT col1 이 없다), 검색이 없고(LIKE '%손흥민%' 불가), 조인·집계가 없다.



분석 층은 새로 만드는 것이 아니다. `footballx/server/backend/data/**/*.*.parquet` 가 이미 그 구조다. DuckDB 만 있으면 SQL 이 그대로 된다.

화면이 Firestore 를 직접 읽어도 되는 경우

- 문서 하나를 주소로 읽을 때 — `matches/{gm_id}/recordings/H`
- 실시간 구독이 필요한 기록 화면 — DID 입력 중의 `records`
- 방금 자기가 쓴 문서를 되읽을 때 — 스냅샷 갱신 전에 화면에 반영하기 위해

관리 화면 - 일정 · 이적시장

읽기는 무겁고 쓰기는 가볍다. 이적은 연 수십~수백 건이라, 쓰기 시점에 스냅샷을 같이 갱신해도 부담이 없다 - "등록했는데 목록에 안 보인다" 문제 자체가 생기지 않는다.

[화면] 검색 · 목록 · 이력 조회 → 파이썬 API → DuckDB 스냅샷
 등록 · 수정 → 파이썬 API → Firestore 트랜잭션 (원본)

13 실데이터 검증 ★

레거시 ff_game_record 실제 1,536행(라운드 2, 20개 팀-경기)을 didLogic.ts 에 수정 없이 넣고 D-MST 값과 대조했다.

항목	결과
DTP 슛 포함 공격루트	20 / 20 팀 일치
GOAL	20 / 20 일치
SHOT	자책골 제외 후 20 / 20 일치
B / M / A / S	17 / 20 일치 (3팀에서 1~2 차이)
공격루트 그룹핑	382개 전부 레거시 <code>gt_id</code> 경계 안. 잘못 합친 것 0건

성능 실측 – "저장하지 않는다"의 근거

레코드	전체 재계산 1회
300건	0.11 ms
600건	0.15 ms
1,200건 한 경기	0.16 ms

한 프레임(16ms) 안에 100번 돌려도 남는다. 계산이 저장보다 압도적으로 싸다.

발견한 버그 – 자책골이 SHOT 에 포함된다

```
// didLogic.ts:415 - 공통 플래그 루프
for (const r of chain) {
  const { act, res } = r // ← 원본 act. isOwnGoal 보장 없음
  f.isSht = isShotAct(act) || !!r.isShot
```

1차·2차 패스는 `isOwnGoal` 을 보장하는데 이 루프만 빠졌다. 자책골을 제외하니 SHOT 이 3팀
불일치 → 20/20 일치로 바뀌었다. 집계 규칙: `GOAL` 은 자책골 포함, `SHOT` 은 제외, `SSR =`
`(GOAL - OG) / SHOT` .

아직 검증되지 않은 것

TACTIC.xlsx에는 슛이 포함된 루트만 들어 있다. 따라서 `gi_tmp` (TAP), `DAP` , `TTP` ,
`BAP` , `ASR` 는 한 번도 실데이터로 검증된 적이 없다. 전체 레코드가 포함된 export 가
필요하다.

14 결정이 필요한 것

240개 컬럼을 훑으면서 갈 곳이 정해지지 않은 것들. 여기가 비어 있으면 규칙 0을 만족한다고 말할 수 없다.

01 ff_game_card – CardDoc 타입 자체가 없다	→ cards 서브컬렉션에 사실 저장. CardDoc 타입·저장 연동 구현
02 gp_point_plus / gp_point_minus – 상단 -3/-1/+1/+3 버튼. 지금 화면에는 버튼만 있고 저장 로직이 없다	→ 원래 무슨 점수였는지 확인 필요
03 pl_in_order – 교체 순번	→ LineupEntry.subOrder 추가 여부

04 gr_area_code_org — 원본 구역코드	→ 진영 반전 전 값으로 추정. 확인 필요
05 gr_shoot_rate_x/y — 골대 내 비율 좌표	→ shootPosX/Y 와 중복인지 확인
06 gi_part_ex — 연장전 진영	→ H3/H4 UI 미구현 상태
07 gm_sub_league — 서브리그	→ 개념 사용 여부
08 t_name_ae / t_name_cn — 아랍어·중국어 팀명	→ 사용 여부
09 최표 단위 — 해결됨. 971×634 픽셀 ↔ 0~100%	→ §05 에 변환식 기재. 실데이터로 검증 완료

그 밖의 미결정

- 기준값을 고정할 것인가, 시즌 평균으로 자동 갱신할 것인가 — 고정하면 과거 평점이 안 변해 재현 가능하다. 자동 갱신은 리그 수준을 따라가지만 **어제 11.65였던 평점이 오늘 11.4가 된다.** 어느 쪽이든 ratingBaseline 버전 스탬프가 있으면 과거 복원은 가능
- 체번 규칙 구현 위치 — Season / M-Code / Match / T-Round의 서버 구현 위치. S-Round는 DID 저장·입력 대상에서 제외하며, 파이썬 출력 단계의 복원·대조 검증을 후속 구현한다(§04)
- 비전 데이터 구조 — 지금은 source / playerIdSource 분기만 남기고 설계는 보류. 필드가 없으면 나중에 과거 레코드에 대해 "사람이 넣었나 기계가 넣었나"를 영영 알 수 없다
- 연장전(H3/H4) — 스키마는 열여줬으나 UI 는 전·후반만
- 확정 후 수정 시 재계산 트리거 — 클라이언트 즉시 재계산 vs 파이썬 위임

레거시 전체 컬럼 · Aimbroad_database_schema-240812.xlsx 의 전체주요목록 시트 (19개 표 240개 컬럼)

컬럼 단위 매핑 원본 · docs/05_legacy_field_mapping.md

판정 로직 규칙 · docs/03_kpi_terminology.md + app/utils/didLogic.ts

타입 정의 · app/types/schema.ts

출력 규칙 · footballx/app/pages/admin/{dMST,playerMST,multiSheet}

대외비 산출식 · repo 밖 별도 관리